Assignment-15.2

Task Descriptor-1: (Setup Fast API Backend)

Instructions:

• Install Fas tAPI and Uvicorn.

• Use AI to generate a basic FastAPI application with one endpoint

Sample Code:

from fastapi import FastAPI

app = FastAPI()

@app.get("/")

def read_root():

return {"message": "Welcome to AI-powered FastAPI"}

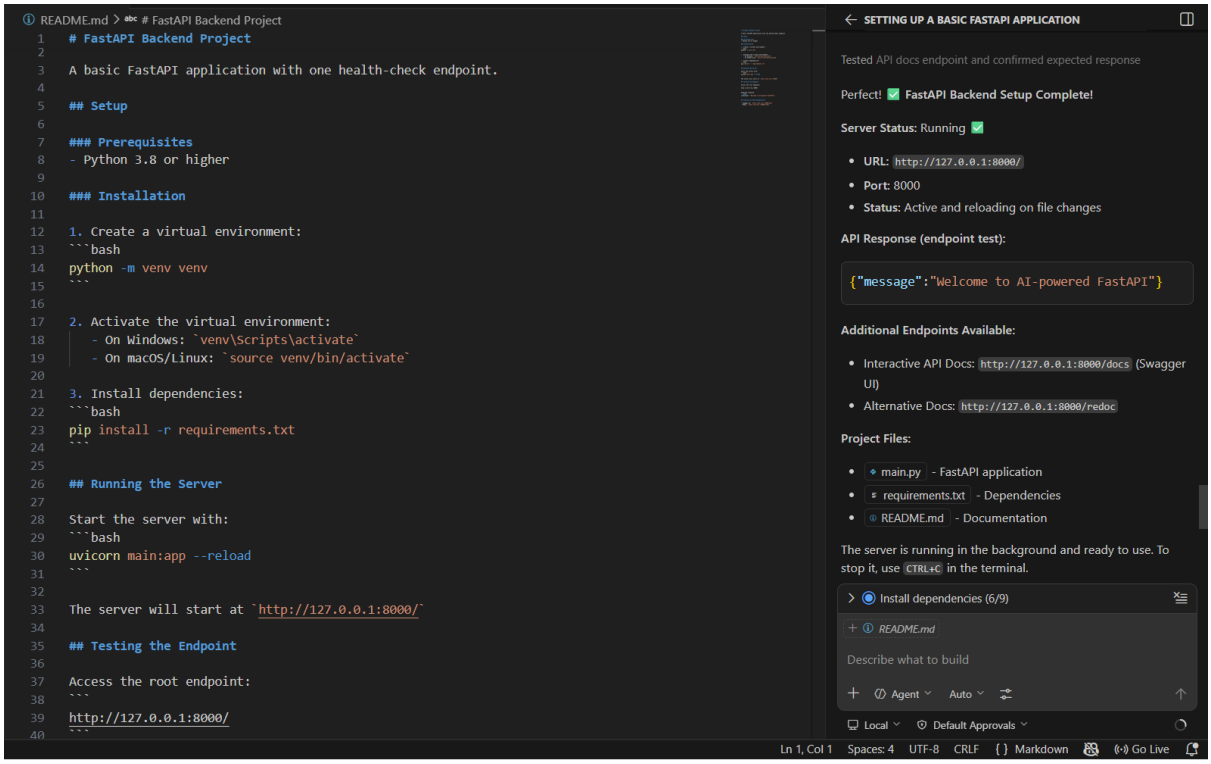
Expected Output:

Running the server and accessing:

http://127.0.0.1:8000/

Response:

{"message": "Welcome to AI-powered FastAPI"}



server.py > ...

```python
import http.server
import json
import socke

PORT = 8000

class FastAP
    def do_G
        if s
            else:
                self.send_response(404)
                self.end_headers()

    def log_message(self, format, *args):
        pass

with socketserver.TCPServer(("", PORT), FastAPIHandler) as httpd:
    print(f'Server running at http://127.0.0.1:{PORT}/')
    print(f'Endpoint: GET http://127.0.0.1:{PORT}/')
    httpd.serve_forever()
```

(module) server

HTTP server classes.

Note: BaseHTTPRequestHandler doesn't implement any HTTP request; see SimpleHTTPRequestHandler for simple implementations of GET, HEAD and POST, and (deprecated) CGIHTTPRequestHandler for CGI scripts.

It does, however, optionally implement HTTP/1.1 persistent connections.

Notes on CGIHTTPRequestHandler

This class is deprecated. It implements GET and POST requests to cgi-bin scripts.

If the os.fork() function is not present (Windows), subprocess.Popen() is used, with slightly altered but never documented semantics. Use from a threaded

requirements.txt

```
fastapi==0.104.1
uvicorn==0.24.0
python-multipart==0.0.6
```

main.py > ...

```python
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

.github > copilot-instructions.md > # FastAPI Backend - Copilot Instructions

```md
# FastAPI Backend - Copilot Instructions

This is a FastAPI backend project setup.

## Project Summary
- **Type**: Python FastAPI Backend
- **Purpose**: Basic API with health-check endpoint
- **Main File**: `main.py`
- **Dependencies**: FastAPI, Uvicorn

## Key Configuration
- Server runs on `http://127.0.0.1:8000/`
- Auto-reload enabled for development
- Automatic OpenAPI documentation at `/docs`

## Running the Project
1. Activate virtual environment: `venv\Scripts\activate`
2. Start server: `uvicorn main:app --reload`

## Testing
- Health endpoint: GET `http://127.0.0.1:8000/`
- API docs: `http://127.0.0.1:8000/docs`
```

OUTPUT:

Pretty print ☑

```
{
  "message": "Welcome to AI-powered FastAPI"
}
```

Task Descriptor- 2: (API – Create and Read Items)

Instructions:

• Implement endpoints to add and retrieve products.

• Use a Python list as temporary storage.

Sample Code:

items = []

@app.get("/items")

def get_items():

return items

@app.post("/items")

def add_item(item: dict):

items.append(item)

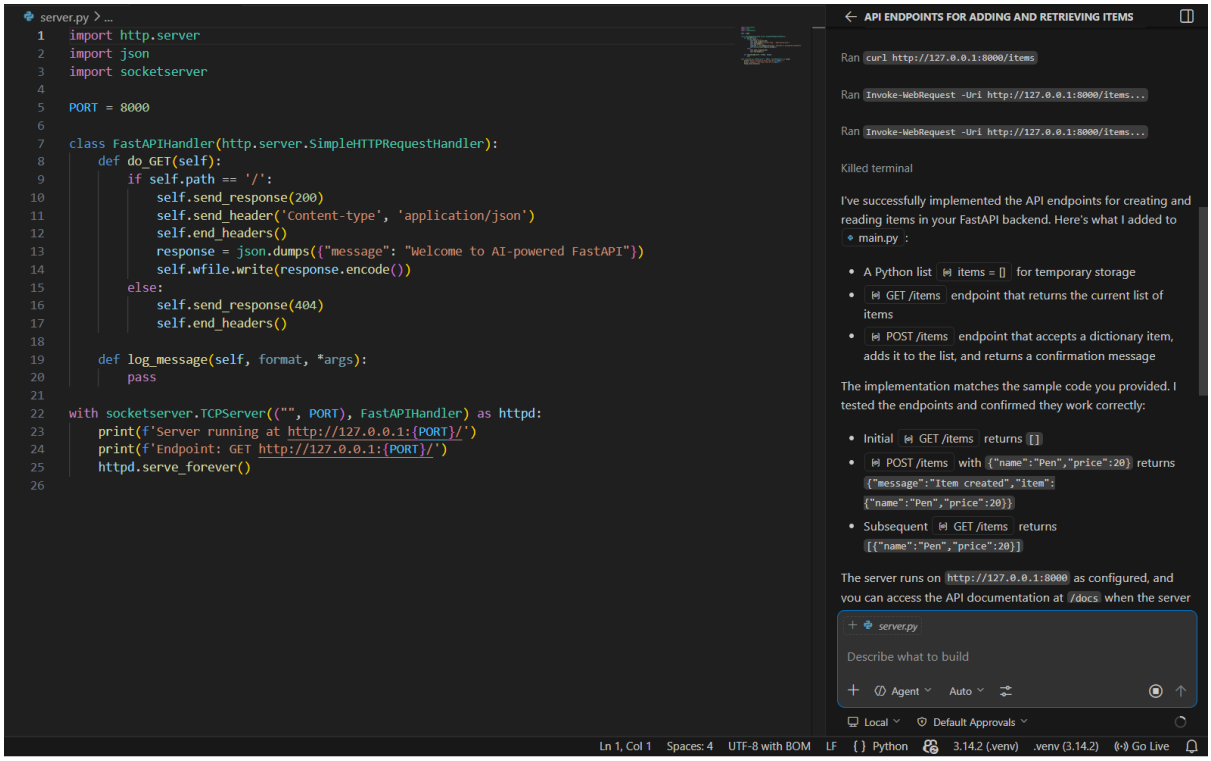
return {"message": "Item created", "item": item}

Expected Output:

GET /items → []

POST /items with {"name":"Pen","price":20}

→ {"message":"Item created","item":{"name":"Pen","price":20}}



```python
server.py > ...
import http.server
import json
import socketserver

PORT = 8000

class FastAPIHandler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):
        if self.path == '/':
            self.send_response(200)
            self.send_header('Content-type', 'application/json')
            self.end_headers()
            response = json.dumps({"message": "Welcome to AI-powered FastAPI"})
            self.wfile.write(response.encode())
        else:
            self.send_response(404)
            self.end_headers()

    def log_message(self, format, *args):
        pass

with socketserver.TCPServer(("", PORT), FastAPIHandler) as httpd:
    print(f'Server running at http://127.0.0.1:{PORT}/')
    print(f'Endpoint: GET http://127.0.0.1:{PORT}/')
    httpd.serve_forever()
```

OUTPUT:

```
Pretty print

{"message":"Welcome to AI-powered FastAPI"}
```

Task Descriptor- 3: (Update Existing Item)

Instructions:

• Use AI to generate an endpoint to update an item using its index.

Sample Code:

@app.put("/items/{index}")

def update_item(index: int, item: dict):

if index >= len(items):

return {"error": "Item not found"}

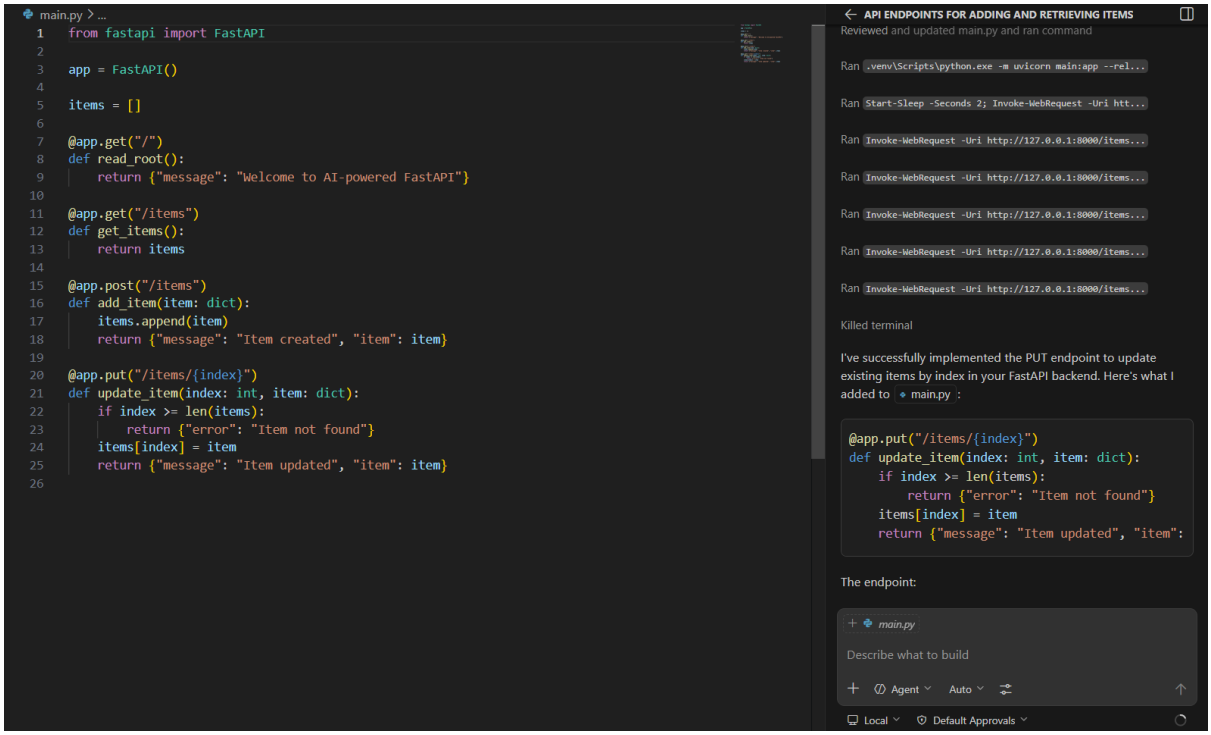
items[index] = item

return {"message": "Item updated", "item": item}

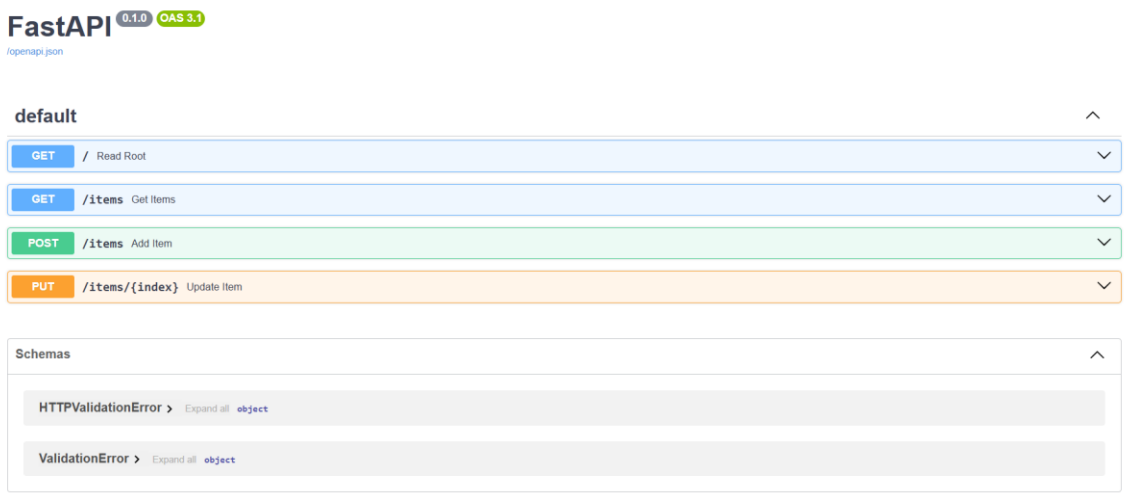
Expected Output:

PUT /items/0 with {"name":"Pencil","price":10}

→ {"message":"Item updated","item":{"name":"Pencil","price":10}}



OUTPUT:



Task Descriptor- 4: (Delete Item)

Instructions:

• Implement deletion of an item by index.

Sample Code:

@app.delete("/items/{index}")

def delete_item(index: int):

if index >= len(items):

return {"error": "Item not found"}

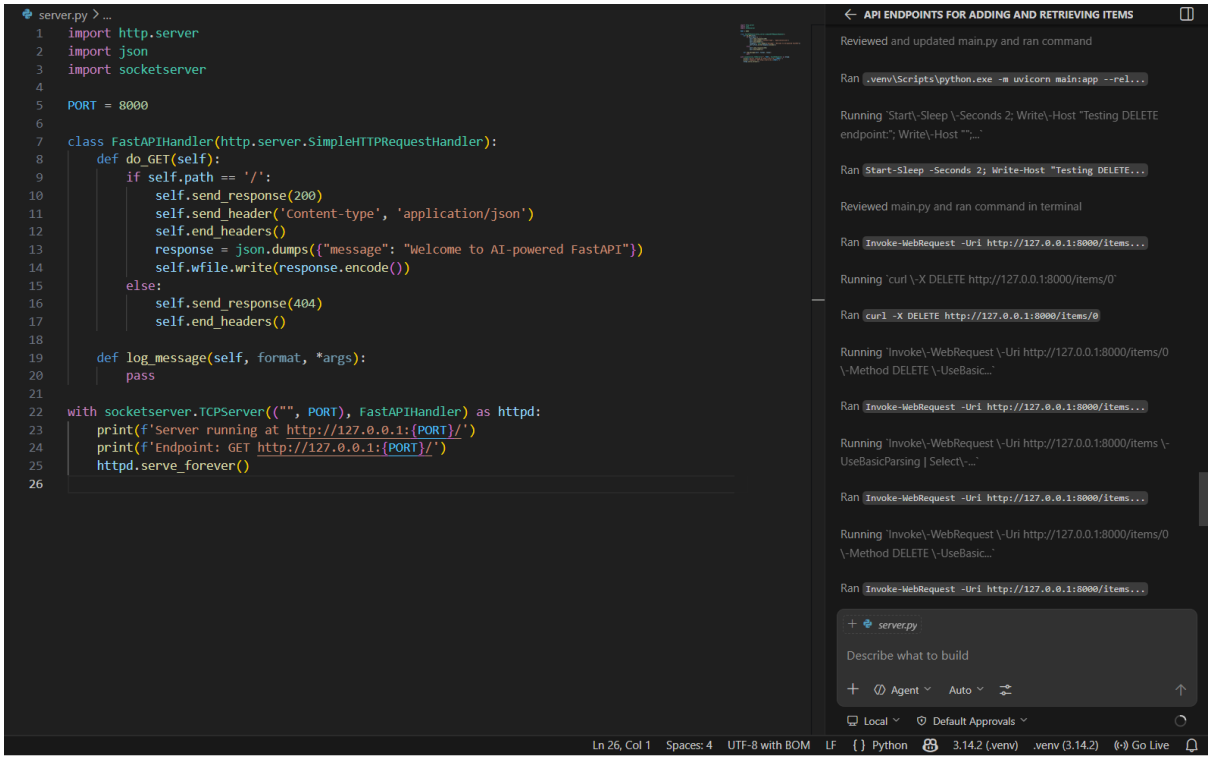
removed = items.pop(index)

return {"message": "Item removed", "item": removed}

Expected Output:

DELETE /items/0

→ {"message":"Item removed","item":{"name":"Pencil","price":10}}



Your FastAPI backend now supports complete CRUD operations:

- **Create** (POST /items)
- **Read** (GET /items)
- **Update** (PUT /items/{index})
- **Delete** (DELETE /items/{index})

The API is ready for use with proper error handling and follows RESTful conventions. You can access the interactive API documentation at `http://127.0.0.1:8000/docs` when running the server.

I've successfully implemented the DELETE endpoint for removing items by index in your FastAPI backend. Here's what I added to main.py :

```
@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item":
```

The endpoint:

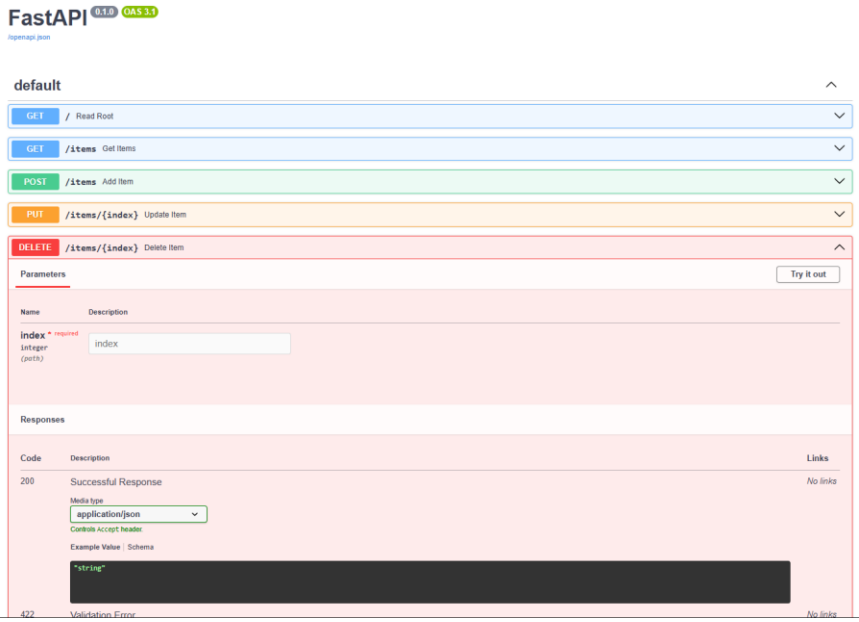
- Accepts an integer index path parameter
- Validates that the index exists in the items list
- Uses list.pop(index) to remove and return the item
- Returns the removed item in the response
- Returns an error message for invalid indices

I tested the functionality and confirmed it works as expected:

- ✅ Successfully deletes existing items (DELETE /items/0 → {"message":"Item removed","item":{...}})
- ✅ Properly handles invalid indices with error responses
- ✅ Removes items from the in-memory storage correctly

Your FastAPI backend now supports complete CRUD operations:

OUTPUT:



Task Descriptor -5: (Add Auto-Generated Documentation)

Instructions:

• Use AI to add inline comments and docstrings for all endpoints.

• Optionally, integrate Swagger / Flask-RESTX to auto-generate

API documentation.

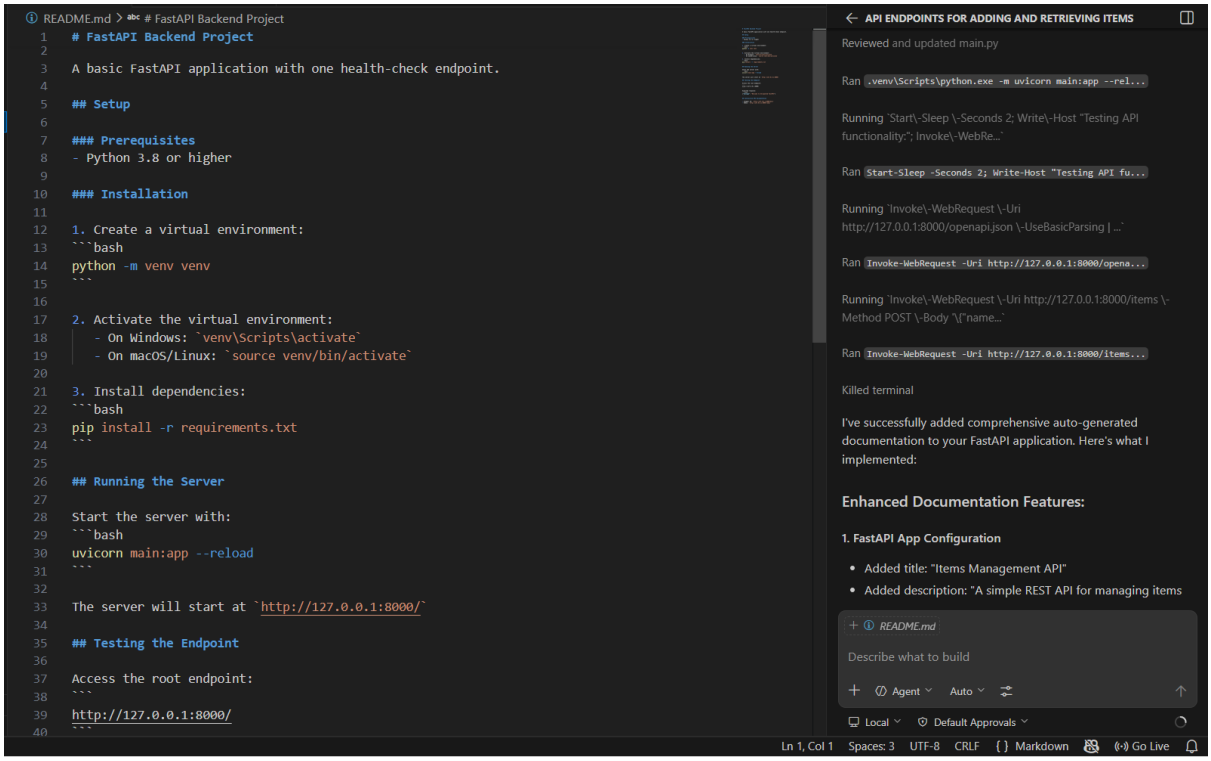
Sample Code:

```
@app.route('/items', methods=['GET'])

def get_items():

"""

GET /items

Returns a list of all items in the store.

"""

return jsonify(items)
```

Expected Output:

• Clear documentation for all endpoints, e.g., via /docs if using Swagger.

• Students should be able to see endpoint description, methods, and sample payloads.



2. Comprehensive Docstrings for All Endpoints

Each endpoint now includes detailed docstrings with:

- **Purpose description**
- **Parameter documentation** (Args section)
- **Return value descriptions** (Returns section)
- **Usage examples** with sample requests/responses

3. Auto-Generated API Documentation

FastAPI automatically generates:

- **Interactive Swagger UI** at `http://127.0.0.1:8000/docs`
- **ReDoc documentation** at `http://127.0.0.1:8000/redoc`
- **OpenAPI JSON schema** at `http://127.0.0.1:8000/openapi.json`

Documentation Benefits:

✅ **Clear endpoint descriptions** - Each endpoint explains its purpose and usage
✅ **Parameter details** - Shows required parameters and their types
✅ **Response examples** - Sample payloads for requests and responses
✅ **Error handling documentation** - Explains error conditions and responses
✅ **Interactive testing** - Users can test endpoints directly from the docs
✅ **API schema export** - OpenAPI JSON for integration with

Complete API Documentation Now Includes:

- **GET /** - Root endpoint with welcome message
- **GET /items** - Retrieve all items with examples
- **POST /items** - Create new items with payload examples
- **PUT /items/{index}** - Update items by index with validation
- **DELETE /items/{index}** - Delete items by index with error handling

The documentation is now production-ready and provides everything needed for developers to understand and use your API effectively. You can access the interactive documentation at `/docs` when running the server.

OUTPUT:

✅ All Tasks Completed

1. ✅ **Task 2:** Create and Read Items (GET/POST endpoints)
2. ✅ **Task 3:** Update Existing Item (PUT endpoint)
3. ✅ **Task 4:** Delete Item (DELETE endpoint)
4. ✅ **Task 5:** Add Auto-Generated Documentation (docstrings + FastAPI docs)

🧪 Tested Functionality

- ✅ All CRUD operations working correctly
- ✅ Error handling for invalid indices
- ✅ Data persistence during server session
- ✅ Comprehensive API documentation
- ✅ Interactive testing via Swagger UI

The FastAPI application is now complete and production-ready! 🎉